

PDF 4 of 4 — Interview Notes: English + Hindi Bilingual

Scalability

Every section of the Interview & Practical Notes (PDF 3) — mental models, trade-offs, misconceptions, interview questions and answers, follow-ups, exercises, and the senior-engineer perspective — translated paragraph by paragraph into Hindi, with a glossary of advanced terms.

पूरा PDF 3 · Bilingual

Q&A + Follow-ups सहित

Advanced Glossary

Contents / विषय-सूची

1. Topic Overview
2. Simple Mental Model
3. Key Points to Remember
4. Real-World Examples
5. When to Use / When NOT to Use
6. Trade-offs
7. Common Misconceptions
8. Interview Questions
9. Interview Answers
10. Follow-up Questions
11. Practical Design Exercises
12. Quick Interview Revision Sheet
13. Senior Engineer Perspective
14. General + Technical Words Glossary

1. Topic Overview

ENGLISH

Scalability is a system's ability to keep working well as demand grows — more users, more requests per second, more data — by adding resources rather than by rewriting everything from scratch. It exists because every system starts on finite hardware, and traffic almost never stays where it started. A service built for 100 users a day eventually meets 100,000 users a day, and the question is whether it bends or breaks.

हिंदी

Scalability किसी सिस्टम की वह क्षमता है जिससे वह बढ़ती हुई demand के साथ भी अच्छी तरह काम करता रहे — चाहे अधिक users हों, प्रति सेकंड अधिक requests हों, या अधिक data हो — और यह सब resources जोड़कर हासिल किया जाता है, न कि सब कुछ शुरू से दोबारा लिखकर। यह इसलिए ज़रूरी है क्योंकि हर सिस्टम सीमित (finite) hardware पर शुरू होता है, और traffic लगभग कभी वहीं नहीं रुकता जहाँ से शुरू हुआ था। एक ऐसी service जो रोज़ 100 users के लिए बनाई गई थी, आखिरकार रोज़ 100,000 users तक पहुँच जाती है, और सवाल यह है कि सिस्टम झुकता (bend) है या टूट (break) जाता है।

ENGLISH

The problem scalability solves is capacity under growth: how do you serve more without a proportional collapse in latency, or without the system falling over completely. There are exactly two levers, and the source lesson names both: make the existing machine bigger (vertical scaling / scaling up), or add more machines and spread the work across them (horizontal scaling / scaling out).

हिंदी

Scalability जिस समस्या को हल करती है वह है growth के दौरान capacity: आप बिना latency में proportional गिरावट के या सिस्टम को पूरी तरह गिरे बिना, ज़्यादा कैसे serve करें। इसके लिए ठीक दो lever हैं, और source lesson दोनों के नाम बताता है: या तो मौजूदा machine को बड़ा बनाएं (vertical scaling / scaling up), या अधिक machines जोड़कर काम को उनमें बाँट दें (horizontal scaling / scaling out)।

ENGLISH

Scalability matters in System Design because it is one of the first things an interviewer probes — it forces you to reveal whether you understand where state lives, where the bottleneck will move to next, and what breaks first under load. It is also foundational: almost every other "-ility" in distributed systems (availability, reliability, fault tolerance) is built on top of a scaling decision. You cannot design for high availability, for instance, without first deciding how the system scales, because availability in a modern system usually comes from having many redundant, horizontally-scaled instances.

हिंदी

System Design में Scalability इसलिए मायने रखती है क्योंकि यह उन पहली चीज़ों में से एक है जिसे interviewer जाँचता (probe) है — यह आपको यह दिखाने पर मजबूर करती है कि क्या आप समझते हैं कि state कहाँ रहता है, bottleneck आगे कहाँ शिफ्ट होगा, और load के तहत सबसे पहले क्या टूटेगा। यह foundational भी है: distributed systems की लगभग हर दूसरी "-ility" (availability, reliability, fault tolerance) किसी न किसी scaling decision पर ही आधारित होती है। उदाहरण के लिए, आप पहले यह तय किए बिना कि सिस्टम कैसे scale करेगा, high availability के लिए design नहीं कर सकते, क्योंकि आधुनिक सिस्टम में availability आमतौर पर कई redundant, horizontally-scaled instances होने से ही आती है।

ENGLISH

In a larger architecture, scalability decisions ripple outward. Choosing horizontal scaling for your application tier immediately creates new requirements: a load balancer to distribute requests, a way to avoid sticky per-instance state, and eventually a data tier that can also scale (through caching, read replicas, or sharding). Choosing vertical scaling defers those requirements — which is sometimes exactly the right call for a small, early-stage system, and sometimes a trap for a system that will need to scale further later.

हिंदी

एक बड़े architecture में, scalability से जुड़े फैसले चारों तरफ असर डालते हैं। अपने application tier के लिए horizontal scaling चुनते ही नई requirements सामने आ जाती हैं: requests बाँटने के लिए एक load balancer, per-instance sticky state से बचने का कोई तरीका, और अंततः एक data tier जो खुद भी scale कर सके (caching, read replicas, या sharding के ज़रिए)। Vertical scaling चुनने से इन requirements को टाला जा सकता है — जो कभी-कभी किसी छोटे, शुरुआती-चरण (early-stage) सिस्टम के लिए बिल्कुल सही फैसला होता है, और कभी-कभी उस सिस्टम के लिए एक जाल (trap) बन जाता है जिसे आगे चलकर और scale करने की ज़रूरत पड़ेगी।

Beginner framing

Scalability = handling more traffic. Two options: bigger machine, or more machines.

शुरुआती समझ: Scalability = अधिक traffic संभालना। दो विकल्प: बड़ी machine, या अधिक machines।

Senior framing

Scalability = a deliberate trade-off between hardware simplicity (vertical) and distributed-systems complexity (horizontal), chosen based on growth rate, availability needs, and how much of the system can be made stateless or partitioned.

Senior समझ: Scalability = hardware की सरलता (vertical) और distributed-systems की जटिलता (horizontal) के बीच एक सोचा-समझा trade-off है, जो growth की रफ़्तार, availability की ज़रूरतों, और सिस्टम को कितना stateless या partitioned बनाया जा सकता है, उस पर आधारित है।

2. Simple Mental Model

The analogy: checkout counters at a supermarket

हिंदी – शीर्षक

उपमा: सुपरमार्केट के checkout counters

ENGLISH

Imagine a supermarket with one checkout counter. When the store gets busy, you have two choices. You can train that one cashier to scan items faster and give them a bigger, faster till — that is **vertical scaling**. Or you can open more checkout counters and let customers queue at whichever one is free — that is **horizontal scaling**.

हिंदी

एक ऐसे सुपरमार्केट की कल्पना कीजिए जिसमें सिर्फ एक checkout counter है। जब स्टोर में भीड़ बढ़ जाती है, तो आपके पास दो विकल्प होते हैं। आप उस एक cashier को items तेज़ी से scan करना सिखा सकते हैं और उसे एक बड़ा, तेज़ till दे सकते हैं — यही **vertical scaling** है। या फिर आप और checkout counters खोल सकते हैं और customers को जो भी counter खाली हो वहाँ queue में लगने दे सकते हैं — यही **horizontal scaling** है।

ENGLISH

The one-cashier approach is simple: there's only one queue, one till, nothing to coordinate. But there's a limit to how fast one person can scan items, and if that cashier goes on a break, the whole store stops selling anything. The many-counters approach has no real ceiling — you can keep opening counters — but now you need someone directing customers to a free counter (a load balancer), and if a customer's loyalty card or shopping cart is tied to "counter 3" specifically, the system breaks the moment counter 3 is busy or down.

हिंदी

एक-cashier वाला तरीका सरल है: सिर्फ एक queue है, एक till है, coordinate करने के लिए कुछ नहीं है। लेकिन एक व्यक्ति कितनी तेज़ी से items scan कर सकता है इसकी एक सीमा है, और अगर वह cashier break पर चला जाए, तो पूरा स्टोर कुछ भी बेचना बंद कर देता है। कई-counters वाले तरीके की कोई असली ceiling नहीं है — आप counters खोलते ही जा सकते हैं — लेकिन अब आपको customers को किसी खाली counter की तरफ भेजने के लिए किसी की ज़रूरत है (एक load balancer), और अगर किसी customer का loyalty card या shopping cart खासतौर पर "counter 3" से जुड़ा है, तो counter 3 के busy या down होते ही सिस्टम टूट जाता है।

Beginner understanding

Vertical = one bigger/faster machine.

Horizontal = many machines sharing the load.

Horizontal is "the scalable one"; vertical "runs out of room."

Vertical = एक बड़ी/तेज़ machine। Horizontal = load बाँटती हुई कई machines। Horizontal को "असली scalable" माना जाता है; vertical की "जगह खत्म हो जाती है।"

Interview-level understanding

Horizontal scaling only pays off if the "customer" (request) doesn't care which "counter" (node) serves it. That requires statelessness at the app tier and a partitioning/replication strategy at the data tier. Without those, adding machines just adds idle machines — the real bottleneck is untouched. This is the gap between "I added servers" and "I actually scaled the system."

Horizontal scaling तभी फायदेमंद होती है जब request को फ़र्क़ न पड़े कि उसे कौन सा node serve कर रहा है। इसके लिए app tier पर statelessness और data tier पर partitioning/replication चाहिए। इनके बिना, machines जोड़ने से सिर्फ़ खाली मशीनें बढ़ती हैं — असली bottleneck जस का तस रहता है। यही फ़र्क़ है "servers जोड़ना" और "वाकई सिस्टम scale करना" के बीच।

3. Key Points to Remember ☆

- ★ **Definition:** Scalability is handling increasing workload by adding resources — either bigger resources (vertical) or more resources (horizontal).
- ★ **Horizontal scaling (scaling out):** add more nodes; workload is distributed across them; the classic approach for distributed systems (e.g. Cassandra, MongoDB).
- ★ **Vertical scaling (scaling up):** increase one node's CPU/memory/storage; simple but bounded by that machine's ceiling.
- ★ **Statelessness is the enabler:** horizontal scaling only works cleanly when any node can serve any request.
- ★ **Vertical scaling usually needs downtime** to resize, and it concentrates everything on one machine — a single point of failure (SPOF).
- ★ **Horizontal scaling needs a load balancer** to actually spread requests across the pool.
- ★ **Diminishing returns exist on both sides:** vertical gets disproportionately expensive at the top; horizontal gets diminishing throughput once a shared resource becomes the bottleneck — this is Amdahl's Law / the Universal Scalability Law in practice.
- ★ **Scaling compute is easier than scaling data.** Stateless app servers scale out almost for free; databases need sharding, partitioning, or replication.
- ★ **Commonly forgotten:** adding more app servers in front of a single, un-partitioned database does not make the system horizontally scalable.
- ★ **Interview point:** "I would scale horizontally" is incomplete. State where the state lives, and what happens on node failure.

हिंदी अनुवाद – मुख्य बिंदु (HINDI TRANSLATION – KEY POINTS)

परिभाषा: Scalability का मतलब है resources जोड़कर बढ़ते हुए workload को संभालना — या तो बड़े resources (vertical) से, या अधिक resources (horizontal) से।

Horizontal scaling (scaling out): अधिक nodes जोड़ना; workload उनमें बाँट दिया जाता है; distributed systems के लिए classic तरीका (जैसे Cassandra, MongoDB)।

Vertical scaling (scaling up): एक node का CPU/memory/storage बढ़ाना; सरल लेकिन उस machine की ceiling तक सीमित।

Statelessness ही इसे संभव बनाता है: horizontal scaling तभी सही से काम करती है जब कोई भी node किसी भी request को serve कर सके।

Vertical scaling को resize के लिए आमतौर पर downtime चाहिए, और यह सब कुछ एक ही machine पर केंद्रित कर देता है — यानी single point of failure (SPOF)।

Horizontal scaling के लिए एक load balancer चाहिए ताकि requests वाकई pool में बँट सकें।

दोनों तरफ diminishing returns होते हैं: ऊपरी छोर पर vertical असमान रूप से महँगी होती है; और shared resource bottleneck बनते ही horizontal से मिलने वाला फ़ायदा घटने लगता है — यही Amdahl's Law / Universal Scalability Law है।

Compute को scale करना data को scale करने से आसान है। Stateless app servers लगभग मुफ्त में scale out हो जाते हैं; databases को sharding, partitioning, या replication चाहिए।

अक्सर भूली जाने वाली बात: एक single, un-partitioned database के आगे और app servers जोड़ने से सिस्टम horizontally scalable नहीं बन जाता।

Interview की बात: सिर्फ "मैं horizontally scale करूँगा" कहना अधूरा जवाब है। यह भी बताएं कि state कहाँ रहता है, और node fail होने पर क्या होता है।

4. Real-World Examples

These are well-known, publicly documented patterns used to illustrate the concept — not confirmed internal architecture of any specific company unless stated as such.

ये well-known, publicly documented patterns हैं जो concept समझाने के लिए इस्तेमाल किए गए हैं — जब तक साफ़ तौर पर न कहा जाए, यह किसी भी company के confirmed internal architecture का बयान नहीं है।

Databases and caches

हिंदी – शीर्षक

Databases और caches

ENGLISH

Cassandra and **MongoDB** (the examples the source lesson itself gives) are built around horizontal scaling: data is partitioned across nodes using a partition/shard key, and adding nodes redistributes data automatically (consistent hashing in Cassandra, chunk-based sharding in MongoDB). **MySQL** and **PostgreSQL** are traditionally scaled vertically first because horizontal write-scaling for a relational database requires extra tooling — read replicas for read scaling, and application-level or proxy-level sharding (e.g. Vitess for MySQL, Citus for PostgreSQL) for write scaling. **Redis** is commonly run as a single powerful instance until Redis Cluster is introduced to shard keys horizontally across nodes.

हिंदी

Cassandra और **MongoDB** (जो source lesson में खुद दिए गए उदाहरण हैं) horizontal scaling को ध्यान में रखकर बनाए गए हैं: data को partition/shard key का उपयोग करके nodes में बाँटा जाता है, और nodes जोड़ने पर data अपने आप फिर से बाँट जाता है (Cassandra में consistent hashing, MongoDB में chunk-based sharding)। **MySQL** और **PostgreSQL** को पारंपरिक रूप से पहले vertically scale किया जाता है, क्योंकि किसी relational database के लिए horizontal write-scaling के लिए अतिरिक्त tooling चाहिए — read scaling के लिए read replicas, और write scaling के लिए application-level या proxy-level sharding (जैसे MySQL के लिए Vitess, PostgreSQL के लिए Citus)। **Redis** को आमतौर पर एक शक्तिशाली single instance के रूप में चलाया जाता है, जब तक keys को horizontally shard करने के लिए Redis Cluster पेश नहीं किया जाता।

Cloud infrastructure

हिंदी – शीर्षक

क्लाउड इन्फ्रास्ट्रक्चर

ENGLISH

AWS EC2 Auto Scaling Groups behind an Application Load Balancer are the textbook implementation of dynamic horizontal scaling. Resizing an **AWS RDS** or **Azure SQL** instance class to get more vCPU/RAM is the textbook implementation of vertical scaling, and it typically requires a brief failover or restart. **GCP** and **Azure** offer equivalent managed autoscaling groups and VM resizing.

हिंदी

किसी Application Load Balancer के पीछे चलने वाले **AWS** EC2 Auto Scaling Groups, dynamic horizontal scaling का textbook उदाहरण हैं। अधिक vCPU/RAM पाने के लिए किसी **AWS RDS** या **Azure SQL** instance class को resize करना, vertical scaling का textbook उदाहरण है, और इसके लिए आमतौर पर एक छोटा failover या restart चाहिए होता है। **GCP** और **Azure** भी समान managed autoscaling groups और VM resizing की सुविधा देते हैं।

Containers and orchestration

हिंदी – शीर्षक

Containers और orchestration

ENGLISH

Kubernetes exposes this exact choice as two separate controllers: the Horizontal Pod Autoscaler (HPA) adds more pod replicas under load, while the Vertical Pod Autoscaler (VPA) increases the CPU/memory request of existing pods. Seeing both exist as first-class, separate features is a good confirmation that this is a genuine architectural fork, not just terminology.

हिंदी

Kubernetes इसी चुनाव को दो अलग-अलग controllers के रूप में सामने रखता है: load बढ़ने पर Horizontal Pod Autoscaler (HPA) अधिक pod replicas जोड़ता है, जबकि Vertical Pod Autoscaler (VPA) मौजूदा pods की CPU/memory request बढ़ा देता है। दोनों का अलग-अलग, first-class features के रूप में मौजूद होना यह पुष्टि करता है कि यह सिर्फ शब्दावली नहीं बल्कि एक असली architectural फ़र्क है।

Products and companies

हिंदी – शीर्षक

प्रोडक्ट्स और कंपनियाँ

ENGLISH

Netflix and **Amazon** are widely known, from public engineering blogs, for running large fleets of horizontally-scaled, stateless microservices behind load balancers — an architecture that assumes individual instances will fail routinely. Consumer platforms with sharp traffic spikes — ride-hailing apps like **Uber** or **Careem**, food delivery apps like **Talabat**, **Zomato**, or **HungerStation**, and e-commerce platforms during sale events (**Flipkart**'s Big Billion Days, **Noon**'s sale days) — are commonly used in interviews as scenario prompts. Payments platforms such as **Razorpay**, **PhonePe**, or **PayTM** are good discussion examples for the "vertical-first, then horizontal" journey on the database side.

हिंदी

Netflix और **Amazon**, अपने public engineering blogs के ज़रिए, load balancers के पीछे horizontally-scaled, stateless microservices का बड़ा fleet चलाने के लिए व्यापक रूप से जाने जाते हैं — यह एक ऐसा architecture है जो यह मान कर चलता है कि individual instances का fail होना सामान्य बात है। अचानक traffic spikes वाले consumer platforms — जैसे **Uber** या **Careem** जैसे ride-hailing apps, **Talabat**, **Zomato**, या **HungerStation** जैसे food delivery apps, और sale events के दौरान e-commerce platforms (**Flipkart** का Big Billion Days, **Noon** के sale days) — interviews में scenario के तौर पर अक्सर इस्तेमाल होते हैं। **Razorpay**, **PhonePe**, या **PayTM** जैसे payments platforms, "पहले vertical, फिर horizontal" वाली journey समझाने के लिए अच्छे उदाहरण हैं।

⚠ Caution / सावधानी

ENGLISH

Treat the company examples above as illustrations of well-known industry patterns, not as verified statements about any company's current, non-public internal architecture.

हिंदी

ऊपर दिए गए company उदाहरणों को well-known industry patterns की मिसाल के रूप में लें, न कि किसी company के मौजूदा, non-public internal architecture के बारे में पुष्टि बयान के रूप में।

5. When to Use / When NOT to Use

When Horizontal Scaling Makes Sense

- Traffic is variable, growing, or hard to predict, and you want to add/remove capacity dynamically.
- High availability matters — you cannot afford a single machine to be a single point of failure.
- The application tier is stateless, or can reasonably be made stateless.
- You want to use commodity hardware instead of paying a premium for the largest machine.
- You need zero-downtime deployments.

हिंदी:

- जब traffic बदलता रहता है, बढ़ रहा है, या अनुमान लगाना मुश्किल है, और capacity को dynamically जोड़ना/हटाना है।
- जब high availability मायने रखती है — एक machine single point of failure नहीं बन सकती।
- जब application tier stateless है, या उसे stateless बनाया जा सकता है।
- जब सबसे बड़ी machine के लिए premium देने के बजाय commodity hardware इस्तेमाल करना है।
- जब zero-downtime deployments चाहिए।

When Horizontal Scaling Is Overkill

- Traffic is small, stable, and well within a single machine's capacity.
- The application is a tightly-coupled legacy monolith where a stateless rewrite is a large risk.
- Team size and operational maturity can't yet support the extra moving parts.
- Licensing is per-core/per-instance and horizontal scaling multiplies cost faster than capacity.
- The workload has a bottleneck that doesn't parallelize.

हिंदी:

- जब traffic छोटा, स्थिर है, और एक machine की capacity में आसानी से आ जाता है।
- जब application एक tightly-coupled legacy monolith है, जिसे stateless बनाना बड़ा जोखिम होगा।
- जब team का आकार और operational maturity अतिरिक्त moving parts संभालने लायक नहीं है।
- जब licensing per-core/per-instance है और horizontal scaling cost को capacity से तेज़ी से बढ़ा देगी।
- जब workload में एक ऐसा bottleneck है जो parallelize नहीं होता।

When Vertical Scaling Makes Sense

- Early-stage systems, low/predictable traffic, or internal tools where simplicity beats theoretical ceiling.
- A quick, low-risk way to buy time before a bigger architectural change.
- Systems that are inherently hard to distribute.
- Reducing operational overhead is more valuable than reducing the theoretical ceiling.

हिंदी:

- शुरुआती चरण के सिस्टम, कम/अनुमानित traffic, या internal tools, जहाँ सरलता ज़्यादा मायने रखती है।
- किसी बड़े architectural बदलाव से पहले समय खरीदने का एक जल्दी, कम-जोखिम वाला तरीका।
- ऐसे सिस्टम जिन्हें distribute करना स्वाभाविक रूप से मुश्किल है।
- जब operational overhead कम करना, theoretical ceiling कम करने से ज़्यादा मूल्यवान है।

When Vertical Scaling Is Risky

- Growth shows no sign of slowing and will likely exceed the largest available machine.
- The system needs high availability — one machine is always a single point of failure.
- Frequent resizing means frequent downtime the business cannot tolerate.

हिंदी:

- जब growth धीमी होने का संकेत नहीं दिखा रही और जल्द ही सबसे बड़ी machine को भी पार कर सकती है।
- जब सिस्टम को high availability चाहिए — एक machine हमेशा single point of failure रहती है।
- जब बार-बार resizing का मतलब है बार-बार downtime, जिसे business बर्दाश्त नहीं कर सकता।

It depends on the requirements / यह requirements पर निर्भर करता है

ENGLISH

There is no universally "correct" choice between horizontal and vertical scaling. The right answer depends on current scale, growth trajectory, availability requirements, team maturity, and cost constraints.

हिंदी

Horizontal और vertical scaling के बीच कोई सार्वभौमिक रूप से "सही" चुनाव नहीं है। सही जवाब मौजूदा scale, growth की दिशा, availability की ज़रूरतों, team की maturity, और cost की सीमाओं पर निर्भर करता है।

6. Trade-offs

Option	Advantages	Disadvantages	Best Use Case
Horizontal Scaling	Near-limitless capacity; no single point of failure; zero-downtime deployments; cheaper commodity instances	Requires statelessness/partitioning; distributed-systems complexity; higher operational overhead	High-traffic, elastic, high-availability systems
Vertical Scaling	Simple; no architecture changes; easy to debug; lower operational overhead	Hard ceiling; single point of failure; resizing needs downtime; non-linear cost at the high end	Small-to-medium systems, legacy monoliths, databases not yet ready to shard

हिंदी अनुवाद — तालिका (Table Translation)

विकल्प (Option)	फ़ायदे (Advantages)	नुकसान (Disadvantages)	सबसे उपयुक्त स्थिति (Best Use Case)
Horizontal Scaling	लगभग असीमित capacity; कोई single point of failure नहीं; zero-downtime deployments; सस्ते commodity instances	Statelessness/partitioning ज़रूरी; distributed-systems की जटिलता; ज़्यादा operational overhead	High-traffic, elastic, high-availability सिस्टम
Vertical Scaling	सरल; कोई architecture बदलाव नहीं; debug करना आसान; कम operational overhead	सख्त ceiling; single point of failure; resize के लिए downtime; ऊपरी सीमा पर non-linear cost	छोटे-मध्यम सिस्टम, legacy monoliths, वे databases जो shard करने के लिए तैयार नहीं

Dimension by dimension

हिंदी — शीर्षक

हर पहलू का विश्लेषण (Dimension by Dimension)

ENGLISH

Performance: vertical scaling can improve single-request latency with zero network overhead; horizontal scaling improves aggregate throughput but each request may now cross a network hop that a single machine wouldn't have.

हिंदी

Performance: Vertical scaling बिना किसी network overhead के single-request latency को बेहतर बना सकती है; जबकि horizontal scaling कुल throughput को बेहतर बनाती है, लेकिन अब हर request को एक network hop पार करना पड़ सकता है जो एक अकेली machine में नहीं होता।

ENGLISH

Scalability: horizontal wins for headroom; vertical wins for how quickly you can apply it without any redesign.

हिंदी

Scalability: headroom के मामले में horizontal जीतती है; बिना redesign के कितनी जल्दी लागू किया जा सकता है, उस मामले में vertical जीतती है।

ENGLISH

Availability & Reliability: horizontal scaling, done right, removes the single point of failure that vertical scaling always carries — but only if the fleet also has redundancy in the data tier.

हिंदी

Availability और Reliability: सही तरीके से की गई horizontal scaling, उस single point of failure को हटा देती है जो vertical scaling में हमेशा मौजूद रहता है — लेकिन तभी, जब data tier में भी redundancy हो।

ENGLISH

Consistency: vertical scaling keeps a single copy of data, so consistency is "free." Horizontal scaling of stateful systems forces an explicit choice — usually framed by the CAP theorem — between strict consistency and availability during network partitions.

हिंदी

Consistency: Vertical scaling data की एक ही copy रखती है, इसलिए consistency "मुफ्त" में मिल जाती है। Stateful systems की horizontal scaling एक स्पष्ट चुनाव करने पर मजबूर करती है — जिसे CAP theorem के ज़रिए समझाया जाता है — network partitions के दौरान strict consistency और availability के बीच।

ENGLISH

Complexity, Operational Overhead & Development Effort: all three are lower for vertical scaling and higher for horizontal scaling — the real price of horizontal scaling.

हिंदी

Complexity, Operational Overhead और Development Effort: ये तीनों vertical scaling में कम और horizontal scaling में ज़्यादा होते हैं — यही horizontal scaling की असली कीमत है।

ENGLISH

Cost: vertical scaling has a steep, non-linear cost curve at the top; horizontal scaling can be cheaper per unit but adds indirect costs — networking, orchestration tooling, engineering time.

हिंदी

Cost: ऊपरी हिस्से में vertical scaling की cost curve तेज़ी से बढ़ती है; horizontal scaling प्रति यूनिट सस्ती हो सकती है, लेकिन इसमें अप्रत्यक्ष लागतें जुड़ जाती हैं — networking, orchestration tooling, engineering का समय।

ENGLISH

Maintainability: a vertically-scaled single instance is simpler to patch and monitor; a horizontally-scaled fleet needs automation to stay maintainable at all.

हिंदी

Maintainability: एक vertically-scaled single instance को patch और monitor करना आसान है; horizontally-scaled fleet को maintainable बनाए रखने के लिए automation चाहिए।

7. Common Misconceptions ⚠️

"Horizontal scaling always gives linear performance gains — 2x machines means 2x throughput."

"Horizontal scaling से हमेशा linear performance फ़ायदा मिलता है — 2x machines का मतलब है 2x throughput।"

WHY IT'S WRONG/INCOMPLETE

Any portion of the workload that can't be parallelized, plus coordination overhead between nodes, caps the real speedup — Amdahl's Law in practice; at very high node counts the Universal Scalability Law shows throughput can even fall.

Workload का जो हिस्सा parallelize नहीं हो सकता, साथ ही nodes के बीच coordination overhead, असली speedup को सीमित कर देते हैं — व्यवहार में यही Amdahl's Law है; बहुत अधिक nodes पर Universal Scalability Law के अनुसार throughput घट भी सकता है।

CORRECT UNDERSTANDING

Horizontal scaling gives diminishing returns unless shared bottlenecks are actively removed.

Horizontal scaling से diminishing returns मिलते हैं, जब तक shared bottlenecks को सक्रिय रूप से न हटाया जाए।

INTERVIEW-LEVEL NUANCE

Be ready to name the bottleneck that will appear next rather than assuming "add more servers" is complete.

"और servers जोड़ दो" को पूरा जवाब मानने के बजाय, यह बताने के लिए तैयार रहें कि अगला bottleneck कहाँ आएगा।

"MongoDB and Cassandra always scale horizontally, with no trade-offs."

"MongoDB और Cassandra हमेशा बिना किसी trade-off के horizontally scale होते हैं।"

WHY IT'S WRONG/INCOMPLETE

Both support horizontal scaling as a capability, not a guarantee. A poor shard/partition key creates hot partitions and expensive cross-shard queries.

दोनों horizontal scaling को एक क्षमता के तौर पर support करते हैं, गारंटी के तौर पर नहीं। गलत shard/partition key से hot partitions और महँगी cross-shard queries बन जाती हैं।

CORRECT UNDERSTANDING

Horizontal scalability in these databases has to be designed for; it doesn't happen automatically.

इन databases में horizontal scalability को सोच-समझकर design करना पड़ता है; यह अपने आप नहीं हो जाता।

INTERVIEW-LEVEL NUANCE

Cassandra and MongoDB also make explicit CAP-theorem trade-offs — horizontal scaling just moves that trade-off into data-modeling decisions.

Cassandra और MongoDB स्पष्ट रूप से CAP-theorem trade-offs भी करते हैं — horizontal scaling इस trade-off को data-modeling के फैसलों में शिफ्ट कर देती है।

"Vertical scaling is outdated and nobody uses it in modern systems."

"Vertical scaling पुरानी बात है और आधुनिक सिस्टम में इसका इस्तेमाल नहीं होता।"

WHY IT'S WRONG/INCOMPLETE

Many production systems still rely on vertical scaling for their primary database, cache, or single-threaded service.

कई production systems आज भी अपने primary database, cache, या single-threaded service के लिए vertical scaling पर निर्भर रहते हैं।

CORRECT UNDERSTANDING

Horizontal and vertical scaling are complementary. A common path: scale vertically first, move to horizontal once forced.

Horizontal और vertical scaling पूरक हैं। एक आम तरीका: पहले vertically scale करें, ज़रूरत पड़ने पर horizontal की ओर बढ़ें।

INTERVIEW-LEVEL NUANCE

Saying "I'd scale vertically first, then re-evaluate" often reads as more senior.

"पहले vertically scale करूँगा, फिर दोबारा आकलन करूँगा" कहना अक्सर ज़्यादा senior जवाब लगता है।

"Stateless just means the service doesn't use a database."

"Stateless का मतलब बस इतना है कि service database इस्तेमाल नहीं करती।"

WHY IT'S WRONG/INCOMPLETE

Statelessness is about where per-request/session context lives, not whether a database is involved.

Statelessness इस बारे में है कि per-request/session context कहाँ रहता है, न कि database शामिल है या नहीं।

CORRECT UNDERSTANDING

A service is stateless when any instance can handle any request because context lives externally (DB, token, cache).

कोई service तभी stateless होती है जब कोई भी instance किसी भी request को संभाल सके क्योंकि context बाहर (DB, token, cache में) रहता है।

INTERVIEW-LEVEL NUANCE

"Stateless app tier, stateful data tier" is the accurate description, not "no state anywhere."

सही वर्णन है "stateless app tier, stateful data tier", न कि "कहीं भी कोई state नहीं है।"

"Adding a load balancer in front of my servers makes the system horizontally scalable."

"अपने servers के आगे load balancer लगा देने से सिस्टम horizontally scalable बन जाता है।"

WHY IT'S WRONG/INCOMPLETE

A load balancer only distributes traffic; if sessions are pinned or every instance shares one un-partitioned database, the bottleneck hasn't moved.

एक load balancer सिर्फ traffic बाँटता है; अगर sessions pinned हैं या हर instance एक ही un-partitioned database से जुड़ा है, तो असली bottleneck नहीं हिला।

CORRECT UNDERSTANDING

True horizontal scalability requires the whole request path to support scale-out.

असली horizontal scalability के लिए पूरे request path का scale-out को support करना ज़रूरी है।

INTERVIEW-LEVEL NUANCE

Trace the request path end-to-end rather than stopping at "we'd add a load balancer."

सिर्फ "हम एक load balancer जोड़ देंगे" पर रुकने के बजाय पूरे request path को शुरू से आखिर तक ट्रेस करें।

8. Interview Questions

ENGLISH

Questions below are organized by difficulty level. Region-relevant company names are noted where the scenario is a natural fit for that market's product companies — this reflects the kind of system those companies operate publicly, not a claim that a specific company asked this exact question.

हिंदी

नीचे दिए गए प्रश्नों को कठिनाई स्तर के अनुसार व्यवस्थित किया गया है। जहाँ scenario उस बाज़ार की product companies के लिए स्वाभाविक रूप से उपयुक्त है, वहाँ region से जुड़े company नाम बताए गए हैं — यह इस बात को दर्शाता है कि वे companies सार्वजनिक रूप से किस तरह का सिस्टम चलाती हैं, यह दावा नहीं कि किसी खास company ने ठीक यही सवाल पूछा था।

L1 Beginner

COMMON INTERVIEW QUESTION

What is scalability in system design, and why does it matter?

System design में scalability क्या है, और यह क्यों मायने रखती है?

COMMON INTERVIEW QUESTION

What is the difference between horizontal scaling and vertical scaling?

Horizontal scaling और vertical scaling में क्या अंतर है?

COMMON INTERVIEW QUESTION

Give one example each of a system that typically scales horizontally and one that typically scales vertically.

एक ऐसे सिस्टम का उदाहरण दीजिए जो आमतौर पर horizontally scale होता है, और एक ऐसे सिस्टम का जो आमतौर पर vertically scale होता है।

FREQUENTLY REPORTED QUESTION

What is a load balancer, and why does horizontal scaling need one?

Load balancer क्या है, और horizontal scaling को इसकी ज़रूरत क्यों पड़ती है?

L2 Intermediate

FREQUENTLY REPORTED QUESTION · India/UAE (Amazon, Flipkart, PhonePe, Careem, Noon)

Why is statelessness important for horizontal scaling, and how do you handle user sessions in a horizontally scaled web application?

Horizontal scaling के लिए statelessness क्यों ज़रूरी है, और horizontally scaled web application में आप user sessions कैसे संभालेंगे?

INTERVIEW-STYLE QUESTION

What challenges would you face trying to horizontally scale a relational database like MySQL, compared to Cassandra or MongoDB?

MySQL जैसे relational database को horizontally scale करने में आपको किन चुनौतियों का सामना करना पड़ेगा, Cassandra या MongoDB की तुलना में?

COMMON INTERVIEW QUESTION

When would you choose vertical scaling over horizontal scaling for a real project?

किसी असली project के लिए आप horizontal scaling के बजाय vertical scaling कब चुनेंगे?

INTERVIEW-STYLE QUESTION

How does Kubernetes' Horizontal Pod Autoscaler differ from its Vertical Pod Autoscaler, and when would you use each?

Kubernetes का Horizontal Pod Autoscaler, Vertical Pod Autoscaler से कैसे अलग है, और आप हर एक का इस्तेमाल कब करेंगे?

L3 Senior

SENIOR-LEVEL FOLLOW-UP

Why doesn't doubling the number of servers always double throughput?

Servers की संख्या दोगुनी करने से throughput हमेशा दोगुना क्यों नहीं होता?

SENIOR-LEVEL FOLLOW-UP · fintech (Razorpay, PhonePe, PayTM, Tabby, Tamara)

How would you design a sharding strategy for a horizontally scaled transactional database, and what goes wrong with a poorly chosen shard key?

आप एक horizontally scaled transactional database के लिए sharding strategy कैसे design करेंगे, और गलत shard key से क्या गड़बड़ होती है?

SENIOR-LEVEL FOLLOW-UP

How do you decide what stays stateless in the application tier versus what must be stateful in the data tier?

आप कैसे तय करते हैं कि application tier में क्या stateless रहेगा, और data tier में क्या ज़रूर stateful होना चाहिए?

SENIOR-LEVEL FOLLOW-UP

What operational complexity does horizontal scaling introduce, and how would you manage it at scale?

Horizontal scaling किस तरह की operational complexity लाती है, और आप इसे बड़े पैमाने पर कैसे manage करेंगे?

L4 Scenario-Based

SCENARIO-BASED QUESTION

Traffic to your API suddenly increases 10x — for example, a flash sale on Flipkart or Noon. What would you change, and in what order?

आपके API पर traffic अचानक 10x बढ़ जाता है — जैसे Flipkart या Noon पर flash sale। आप क्या बदलेंगे, और किस क्रम में?

SCENARIO-BASED QUESTION

One node in your horizontally scaled fleet fails in the middle of the night. Walk through exactly what happens.

आधी रात को आपके horizontally scaled fleet का एक node fail हो जाता है। बताइए कि बिल्कुल क्या होता है।

SCENARIO-BASED QUESTION

Your database has become the bottleneck even though your application servers have plenty of headroom. What would you do?

आपका database bottleneck बन गया है, भले ही application servers के पास काफ़ी गुंजाइश है। आप क्या करेंगे?

SCENARIO-BASED QUESTION

How would you make a currently single-machine, vertically-scaled service highly available without a full rewrite?

एक machine पर vertically-scaled service को बिना पूरी तरह दोबारा लिखे आप highly available कैसे बनाएँगे?

SCENARIO-BASED QUESTION · Talabat, Zomato, Swiggy, HungerStation, Jahez

A food-delivery order service sees predictable but sharp spikes at lunch and dinner. How would you reduce latency and avoid over-provisioning?

एक food-delivery order service में lunch और dinner के समय तेज़ spikes आते हैं। over-provisioning से बचते हुए आप latency कैसे कम करेंगे?

9. Interview Answers ☆

Q: What's the real difference between horizontal and vertical scaling, and how do you decide between them?

Q (हिंदी): Horizontal और vertical scaling में असली फ़र्क क्या है, और आप इनके बीच फैसला कैसे करते हैं?

MODEL ANSWER

Horizontal scaling adds machines and spreads the workload; vertical scaling makes one machine bigger. I'd default to vertical scaling early, because it needs no architecture change — but I'd switch to horizontal scaling once I need high availability, once traffic growth is likely to exceed the largest available instance, or once I need to deploy without downtime. The trade-off is that horizontal scaling only pays off if the app tier is stateless and the data tier can be replicated or partitioned.

Horizontal scaling machines जोड़ती है और workload को बाँट देती है; vertical scaling एक machine को बड़ा बना देती है। मैं शुरुआत में vertical scaling को default मानूँगा, क्योंकि इसके लिए architecture बदलने की ज़रूरत नहीं होती — लेकिन जैसे ही मुझे high availability चाहिए हो, traffic growth सबसे बड़े instance को पार करने वाली हो, या बिना downtime के deploy करना हो, मैं horizontal scaling पर स्विच कर दूँगा। इसका trade-off यह है कि horizontal scaling तभी फ़ायदेमंद है जब app tier stateless हो और data tier को replicate या partition किया जा सके।

Q: Why is statelessness important, and how do you handle sessions?

Q (हिंदी): Statelessness क्यों ज़रूरी है, और आप sessions कैसे संभालते हैं?

MODEL ANSWER

Horizontal scaling assumes any node can handle any request. If session data lives only in one instance's memory, requests from that user must always return to that instance, which defeats load balancing. I'd move session state to a shared store — a cache like Redis, or a signed token (JWT) — so the app tier stays stateless.

Horizontal scaling यह मान कर चलती है कि कोई भी node किसी भी request को संभाल सकता है। अगर session data सिर्फ एक instance की memory में रहता है, तो उस user की requests को हमेशा उसी instance पर वापस जाना होगा, जिससे load balancing बेकार हो जाती है। मैं session state को किसी shared store में ले जाऊँगा — जैसे Redis, या एक signed token (JWT) — ताकि app tier stateless बना रहे।

Q: Traffic suddenly increases 10x. What would you change?

Q (हिंदी): Traffic अचानक 10x बढ़ जाता है। आप क्या बदलेंगे?

MODEL ANSWER

First I'd confirm the app tier is stateless and put it behind auto-scaling. Next I'd look at the database: add a cache in front of hot read paths, and read replicas if reads dominate. If writes are the problem, I'd only reach for sharding once caching and replicas aren't enough.

सबसे पहले मैं यह पक्का करूँगा कि app tier stateless है और उसे auto-scaling के पीछे लगाऊँगा। इसके बाद database देखूँगा: hot read paths के आगे एक cache जोड़ूँगा, और अगर reads हावी हैं तो read replicas जोड़ूँगा। अगर writes समस्या हैं, तो caching और replicas काफ़ी न होने पर ही मैं sharding अपनाऊँगा।

Requirement: sudden, very high read traffic / अचानक, बहुत ज़्यादा read traffic

Decision: introduce caching in front of the database / database के आगे caching लागू करना

Reason: reduces database load and latency fastest / सबसे तेज़ी से database load और latency कम करता है

Trade-off: cache invalidation and stale data / cache invalidation और stale data

Alternative: add read replicas instead of, or alongside, a cache / cache की जगह या साथ में read replicas जोड़ना

Q: The database has become the bottleneck. What would you do?

Q (हिंदी): Database bottleneck बन गया है। आप क्या करेंगे?

MODEL ANSWER

I'd first find out whether it's a read problem or a write problem. For reads, caching and read replicas usually solve it. For writes, I'd vertically scale the instance first; if that ceiling is close, I'd consider partitioning/sharding by a key that matches the access pattern.

मैं सबसे पहले पता करूँगा कि यह read की समस्या है या write की। Reads के लिए, caching और read replicas आमतौर पर समस्या सुलझा देते हैं। Writes के लिए, मैं पहले database instance को vertically scale करूँगा; अगर वह ceiling करीब है, तो access pattern से मेल खाती key के आधार पर partitioning/sharding पर विचार करूँगा।

Q: How would you design a sharding strategy, and what goes wrong with a bad shard key?

Q (हिंदी): आप sharding strategy कैसे design करेंगे, और गलत shard key से क्या गड़बड़ होती है?

MODEL ANSWER

I'd pick a shard key that matches the dominant access pattern and spreads load evenly. A bad shard key creates hot partitions, where one shard absorbs most of the traffic. I'd also plan for resharding up front.

मैं एक ऐसा shard key चुनूँगा जो मुख्य access pattern से मेल खाता हो और load को समान रूप से बाँट दे। एक गलत shard key hot partitions बना देता है, जहाँ एक shard ज़्यादातर traffic खींच लेता है। मैं शुरुआत में ही resharding की योजना भी बनाऊँगा।

Q: One node fails. What happens?

Q (हिंदी): एक node fail हो जाता है। क्या होता है?

MODEL ANSWER

It depends entirely on the architecture. In a horizontally-scaled, stateless fleet with health checks, traffic reroutes and a replacement instance comes up automatically. In a vertically-scaled setup, that failure means downtime until it's restored.

यह पूरी तरह architecture पर निर्भर करता है। Health checks वाले horizontally-scaled, stateless fleet में, traffic फिर से भेज दिया जाता है और एक replacement instance अपने आप खड़ा हो जाता है। एक vertically-scaled setup में, उस failure का मतलब है downtime, जब तक उसे बहाल नहीं किया जाता।

10. Follow-up Questions 🔥

Chain 1 — Choosing horizontal scaling

Chain 1 — Horizontal Scaling चुनना

INTERVIEWER

"Why did you choose horizontal scaling?"

"आपने horizontal scaling क्यों चुनी?"

INTERVIEWER

"Why can't you use vertical scaling?"

"आप vertical scaling का इस्तेमाल क्यों नहीं कर सकते?"

INTERVIEWER

"What happens if one instance fails?"

"अगर एक instance fail हो जाए तो क्या होता है?"

INTERVIEWER

"How do you manage session state?"

"आप session state कैसे manage करते हैं?"

INTERVIEWER

"What becomes the bottleneck after adding more servers?"

"और servers जोड़ने के बाद अगला bottleneck क्या बनता है?"

INTERVIEWER

"How would you scale the database?"

"आप database को कैसे scale करेंगे?"

Chain 2 — Sharding the database

Chain 2 — Database को Shard करना

INTERVIEWER

"Why did you choose to shard the database instead of just adding read replicas?"

"सिर्फ read replicas जोड़ने के बजाय आपने database को shard करना क्यों चुना?"

INTERVIEWER

"How did you pick the shard key?"

"आपने shard key कैसे चुनी?"

INTERVIEWER

"What happens when one shard becomes a hotspot?"

"जब एक shard hotspot बन जाए तो क्या होता है?"

INTERVIEWER

"How do you handle a query that needs data from two different shards?"

"जिस query को दो अलग-अलग shards के data की ज़रूरत हो, उसे आप कैसे संभालते हैं?"

INTERVIEWER

"How do you rebalance data when you add a new shard?"

"नया shard जोड़ने पर आप data को कैसे rebalance करते हैं?"

Chain 3 — Vertical scaling as the first move

Chain 3 — पहला कदम के तौर पर Vertical Scaling

INTERVIEWER

"You said you'd scale vertically first — why not go straight to horizontal?"

"आपने कहा कि आप पहले vertically scale करेंगे — सीधे horizontal पर क्यों नहीं गए?"

INTERVIEWER

"What signal would tell you it's time to switch to horizontal scaling?"

"कौन सा संकेत बताएगा कि horizontal scaling पर स्विच करने का समय आ गया है?"

INTERVIEWER

"What happens to your users during the resize/downtime window?"

"Resize/downtime के दौरान आपके users का क्या होता है?"

INTERVIEWER

"How would you reduce that downtime without redesigning the whole system yet?"

"पूरे सिस्टम को अभी दोबारा design किए बिना आप उस downtime को कैसे कम करेंगे?"

11. Practical Design Exercise

Exercise 1 — Scalable Link-Shortening API

Design a link-shortening service (like a simplified Bitly) that starts with modest traffic but needs to support rapid, unpredictable growth in read traffic.

अभ्यास 1 — Scalable Link-Shortening API: एक link-shortening service design कीजिए (जैसे एक simplified Bitly) जो मामूली traffic के साथ शुरू होती है लेकिन जिसे read traffic में तेज़, अप्रत्याशित growth को संभालना है।

Identify: requirements, expected traffic, storage needs, architecture, likely bottlenecks, scaling strategy, failure scenarios, and trade-offs — before reading further.

आगे पढ़ने से पहले पहचानिए: requirements, अनुमानित traffic, storage की ज़रूरतें, architecture, संभावित bottlenecks, scaling strategy, failure scenarios, और trade-offs।

Expected Thinking Process / अपेक्षित सोच-प्रक्रिया

- Clarify read:write ratio — redirects will vastly outnumber shortenings, so optimize reads first.
- Application tier: keep it stateless so it can scale horizontally from day one.
- Data tier: the mapping is simple key-value data — a strong candidate for a cache.
- Think about the write path separately: avoid ID-generation collisions across instances.
- Consider failure: cache down, app instance dies, database temporarily unreachable.

- Read:write अनुपात स्पष्ट करें — redirects, shortenings से कहीं ज़्यादा होंगे, इसलिए पहले reads को optimize करें।
- Application tier को stateless रखें ताकि यह शुरू से ही horizontally scale कर सके।
- Data tier की mapping एक साधारण key-value data है — cache के लिए एक मज़बूत उम्मीदवार।
- Write path के बारे में अलग से सोचें: कई instances में ID-generation collisions से बचें।
- Failure पर विचार करें: cache down हो, app instance मर जाए, database अस्थायी रूप से unreachable हो।

Reference Solution / संदर्भ समाधान

Stateless application servers behind a load balancer, auto-scaled on request volume. A cache sits in front of the database for the hot path. The database starts as a single, vertically-scaled instance with a replica; if it outgrows one instance, shortcodes can be partitioned by a hash of the code itself. Short-code generation avoids collisions using pre-allocated ranges or a distributed ID scheme.

Load balancer के पीछे stateless application servers, जो request volume के आधार पर auto-scaled हों। Hot path के लिए database के आगे एक cache लगाया जाता है। Database शुरूआत में एक single, vertically-scaled instance होता है जिसके साथ एक replica हो; अगर यह एक instance से बड़ा हो जाए, तो shortcodes को code के hash के आधार पर partition किया जा सकता है। Short-code generation, pre-allocated ranges या distributed ID scheme इस्तेमाल करके collisions से बचता है।

Exercise 2 — Vertically-Scaled Order Service Hitting Its Ceiling

A food-delivery order service has run on a single, vertically-scaled MySQL server for two years and has now hit the largest available instance size, with write latency degrading at peak. Redesign it.

अभ्यास 2 — अपनी Ceiling तक पहुँच चुकी Order Service: एक food-delivery order service दो साल से एक ही vertically-scaled MySQL server पर चल रही है और अब सबसे बड़े instance size पर है, जहाँ peak समय पर write latency बिगड़ जाती है। इसे दोबारा design कीजिए।

Identify: requirements, traffic pattern, storage, architecture, bottlenecks, scaling strategy, failure scenarios, and trade-offs — before reading further.

आगे पढ़ने से पहले पहचानिए: requirements, traffic pattern, storage, architecture, bottlenecks, scaling strategy, failure scenarios, और trade-offs।

Expected Thinking Process / अपेक्षित सोच-प्रक्रिया

- Recognize the traffic pattern is predictable and spiky, not constantly growing.
- Separate read load from write load — they need different fixes.
- "Biggest instance size" means vertical scaling is exhausted — the next lever is horizontal.
- Consider whether writes need to hit the database synchronously, or can be smoothed with a queue.
- Think about a good shard key (restaurant ID or region) matching the query pattern.

- पहचानें कि traffic pattern अनुमानित और spiky है, लगातार बढ़ने वाला नहीं।
- Read load को write load से अलग करें — इनके समाधान अलग-अलग हैं।
- "सबसे बड़ा instance size" का मतलब है vertical scaling खत्म — अगला लीवर horizontal है।
- विचार करें कि writes को database पर synchronously पड़ना ज़रूरी है, या queue से smooth किया जा सकता है।
- query pattern से मेल खाती एक अच्छी shard key (restaurant ID या region) के बारे में सोचें।

Reference Solution / संदर्भ समाधान

Move read-heavy traffic off the primary database: add a cache for menu data and read replicas for order-status reads. Introduce a message queue between order placement and processing to smooth write bursts. Shard orders by region or restaurant ID, since most queries are naturally scoped that way.

Read-heavy traffic को primary database से हटा दें: menu data के लिए cache और order-status reads के लिए read replicas जोड़ें। Order देने और process होने के बीच एक message queue लगाकर write बर्स्ट को smooth करें। Orders को region या restaurant ID के हिसाब से shard करें, क्योंकि ज़्यादातर queries स्वाभाविक रूप से उसी दायरे में आती हैं।

12. Quick Interview Revision Sheet

30-Second Revision

Scalability = handling more work by adding resources. Horizontal = more machines, near-limitless, needs statelessness + a load balancer, no SPOF. Vertical = a bigger machine, simple, hard ceiling, usually needs downtime, single point of failure.

Scalability = resources जोड़कर ज़्यादा काम संभालना। Horizontal = अधिक machines, लगभग असीमित, statelessness + load balancer चाहिए, कोई SPOF नहीं। Vertical = एक बड़ी machine, सरल, सख्त ceiling, आमतौर पर downtime चाहिए, single point of failure।

2-Minute Revision

- Two directions: horizontal (scaling out) and vertical (scaling up).
- Horizontal scaling requires a stateless app tier, a load balancer, and a scalable data tier.
- Vertical scaling needs zero redesign but has a hard ceiling and is a single point of failure.
- Databases are harder to scale horizontally than stateless app servers.
- Horizontal scaling gives diminishing returns once a shared bottleneck remains (Amdahl's Law).
- A good answer always names where state lives and what the next bottleneck will be.
- Real systems commonly scale vertically first, then move to horizontal.

- दो दिशाएँ: horizontal (scaling out) और vertical (scaling up)।
- Horizontal scaling के लिए एक stateless app tier, एक load balancer, और एक scalable data tier चाहिए।
- Vertical scaling के लिए redesign नहीं चाहिए, लेकिन इसकी एक सख्त ceiling है और यह single point of failure है।
- Databases को horizontally scale करना stateless app servers से मुश्किल है।
- Shared bottleneck बने रहने पर horizontal scaling से diminishing returns मिलते हैं (Amdahl's Law)।
- एक अच्छा जवाब हमेशा बताता है कि state कहाँ रहता है और अगला bottleneck क्या होगा।
- असली सिस्टम आमतौर पर पहले vertically scale होते हैं, फिर horizontal की ओर बढ़ते हैं।

Interview Cheat Sheet

	Horizontal	Vertical
Change	# of machines	Size of one machine
Ceiling	Practically far away	Biggest instance available
Downtime to add capacity	No	Usually yes
Single point of failure	No (if redundant)	Yes
Prerequisite	Statelessness / partitioning	None
Complexity / ops overhead	Higher	Lower

हिंदी अनुवाद

	Horizontal	Vertical
बदलाव	Machines की संख्या	एक machine का आकार
Ceiling (सीमा)	व्यावहारिक रूप से बहुत दूर	सबसे बड़ा उपलब्ध instance
Capacity जोड़ने पर downtime	नहीं	आमतौर पर हाँ
Single Point of Failure	नहीं (अगर redundant हो)	हाँ
ज़रूरी शर्त	Statelessness / Partitioning	कुछ नहीं
Complexity / Ops Overhead	ज़्यादा	कम

13. Senior Engineer Perspective

How a Senior Engineer Should Think About This

ENGLISH

An engineer with 7–12 years of experience doesn't start from "horizontal is modern, vertical is legacy." They start from requirements: current traffic, growth trajectory, availability SLA, team size, and budget. Vertical scaling is often the right first move precisely because it costs nothing in engineering time — a senior engineer isn't afraid to recommend the "boring" option when the data doesn't justify the complex one yet.

हिंदी

7–12 साल के अनुभव वाला engineer यह सोचकर शुरुआत नहीं करता कि "horizontal आधुनिक है, vertical पुराना है।" वह requirements से शुरुआत करता है: मौजूदा traffic, growth की दिशा, availability SLA, team का आकार, और बजट। Vertical scaling अक्सर सही पहला कदम होता है, ठीक इसलिए क्योंकि इसमें engineering समय की कोई लागत नहीं लगती — एक senior engineer "boring" विकल्प सुझाने से नहीं डरता, जब data अभी जटिल विकल्प को सही नहीं ठहराता।

ENGLISH

What changes with seniority is the ability to see the second-order effects. Choosing horizontal scaling isn't just "add more servers" — it's accepting an ongoing tax: service discovery, deployment orchestration, distributed tracing and centralized logging, consistent configuration across a fleet, and a security model where every instance-to-instance call needs its own authentication. A senior engineer weighs that tax against the alternative — staying vertical and accepting a lower ceiling and an SLA-limiting single point of failure.

हिंदी

Seniority के साथ जो चीज़ बदलती है वह है second-order प्रभावों को देखने की क्षमता। Horizontal scaling चुनना सिर्फ "और servers जोड़ना" नहीं है — यह एक लगातार चलने वाला 'tax' स्वीकार करना है: service discovery, deployment orchestration, distributed tracing और centralized logging, पूरे fleet में एक जैसी configuration, और एक security model जहाँ हर instance-to-instance call को अपना authentication चाहिए। एक senior engineer इस tax को दूसरे विकल्प के मुकाबले तौलता है — यानी vertical बने रहना और एक कम ceiling तथा SLA को सीमित करने वाला single point of failure स्वीकार करना।

ENGLISH

Failure modes get equal weight to the happy path. For a horizontally-scaled system: what happens when a node fails mid-request, when health checks lag reality, when auto-scaling reacts too slowly, or when a "stateless" service has a subtle sticky dependency. For a vertically-scaled system: what the actual recovery time objective is when that one machine goes down.

हिंदी

Failure modes को happy path जितना ही महत्व दिया जाता है। किसी horizontally-scaled सिस्टम के लिए: क्या होता है जब कोई node request के बीच में fail हो जाए, जब health checks असली स्थिति से पीछे रह जाएँ, जब auto-scaling किसी spike पर धीरे react करे, या जब किसी "stateless" service में कोई छुपी हुई sticky dependency निकल आए। किसी vertically-scaled सिस्टम के लिए: जब वह एक machine down हो जाए तो असली recovery time objective क्या है।

ENGLISH

Cost is treated as an engineering input, not an afterthought. Observability is treated as a prerequisite for scaling out, not a nice-to-have added later. And maintainability is judged by whether the team can operate the result at 3 a.m. without the one person who built it.

हिंदी

Cost को एक बाद की सोच नहीं, बल्कि एक engineering input माना जाता है। Observability को बाद में जोड़ी जाने वाली अच्छी चीज़ नहीं, बल्कि scaling out के लिए एक ज़रूरी शर्त माना जाता है। और maintainability को इस आधार पर आँका जाता है कि क्या team उस नतीजे को रात 3 बजे भी, बिना उस एक व्यक्ति के जिसने इसे बनाया था, चला सकती है।

The senior-level default / Senior-level डिफ़ॉल्ट

ENGLISH

Never present this as "always scale horizontally" or "vertical scaling is outdated." The correct senior answer is always: it depends on the requirements — and then show the reasoning: Requirement → Decision → Reason → Trade-off → Alternative.

हिंदी

इसे कभी भी "हमेशा horizontally scale करो" या "vertical scaling पुरानी बात है" के रूप में पेश न करें। सही senior जवाब हमेशा यही होता है: यह requirements पर निर्भर करता है — और फिर यह तर्क दिखाएं: Requirement → Decision → Reason → Trade-off → Alternative।

General + Technical Words Glossary

Advanced words and phrases used across this Interview Notes document — to build both your professional English and System Design vocabulary.

General English Words

English Word / Term	Hindi Meaning	Simple Explanation
demand	माँग	The amount of something (e.g. traffic) that users want.
finite	सीमित	Having a limit; not infinite.
trajectory	दिशा / रुझान	The path or direction something is heading in, e.g. growth trajectory.
lever	लीवर / उपाय	A tool or action used to produce a specific effect.
foundational	बुनियादी	Forming the base that other things are built on.
ripple outward	असर फैलना	Effects spreading out from one decision to many others.
sticky	चिपकी हुई	Tied to one specific place/instance and hard to move.
defer	टालना	To put off or delay something to later.
trap	जाल	A situation that looks fine now but causes problems later.
diminishing returns	घटते हुए फ़ायदे	Getting less and less benefit for the same extra effort.
disproportionately	असमान रूप से	Growing at an unfair or unequal rate compared to something else.
headroom	गुंजाइश	Extra capacity available before hitting a limit.
overkill	ज़रूरत से ज़्यादा	Doing far more than what is actually needed.
tightly-coupled	कसकर जुड़ा हुआ	Parts of a system that heavily depend on each other.
hotspot / hot partition	हॉटस्पॉट	A part of the system that gets unfairly high load.
foreseeable horizon	निकट भविष्य	A time frame that can be reasonably predicted.
burst	उछाल	A sudden, short increase in activity or traffic.
over-provisioning	ज़रूरत से ज़्यादा संसाधन देना	Allocating more resources than actually needed most of the time.

Technical Words

English Word / Term	Hindi Meaning	Simple Explanation
Amdahl's Law	एमडाल का नियम	The non-parallel part of a task limits total speedup from adding more machines.
Universal Scalability Law	यूनिवर्सल स्केलेबिलिटी लॉ	A model showing throughput can even fall at very high concurrency.
CAP Theorem	CAP प्रमेय	A distributed system can't fully guarantee Consistency, Availability, and Partition tolerance together.
Sharding	शार्डिंग	Splitting a database's data across multiple machines based on a key.
Shard Key / Partition Key	शार्ड / पार्टिशन की	The field used to decide which shard a piece of data belongs to.
Resharding	रीशार्डिंग	Redistributing data across shards after adding or removing shards.
Auto-Scaling	ऑटो-स्केलिंग	Automatically adding or removing instances based on load.
Load Balancer	लोड बैलेंसर	Distributes incoming traffic across multiple servers.
Horizontal Pod Autoscaler (HPA)	होराइज़ॉन्टल पॉड ऑटोस्केलर	A Kubernetes feature that adds more pod replicas under load.
Vertical Pod Autoscaler (VPA)	वर्टिकल पॉड ऑटोस्केलर	A Kubernetes feature that increases a pod's CPU/ memory allocation.
Read Replica	रीड रेप्लिका	A copy of a database used to handle read traffic separately.
Connection Pool	कनेक्शन पूल	A managed set of reusable database connections shared across requests.
Message Queue	मैसेज क्यू	Holds tasks so they can be processed at a sustainable rate.
Orchestration	ऑर्केस्ट्रेशन	Automated coordination and management of many services/containers.
Service Discovery	सर्विस डिस्कवरी	Lets services find and connect to each other automatically.

English Word / Term	Hindi Meaning	Simple Explanation
Observability	ऑब्ज़र्वेबिलिटी	The ability to understand a system's state from logs, metrics, and traces.
Distributed Tracing	डिस्ट्रिब्यूटेड ट्रेसिंग	Tracking a single request as it moves across multiple services.
SLA (Service Level Agreement)	SLA (सर्विस लेवल एग्रीमेंट)	A commitment about a system's expected availability or performance.
RTO (Recovery Time Objective)	RTO (रिकवरी टाइम ऑब्जेक्टिव)	The target time to restore a system after a failure.
Consistent Hashing	कंसिस्टेंट हैशिंग	A hashing technique minimizing data movement when nodes change.
Chaos Engineering	केऑस इंजीनियरिंग	Deliberately introducing failures to test a system's resilience.
Failover	फेलओवर	Automatically switching to a backup system when the primary fails.